TRVL SPRINT PLAN

# Integrated 3-Hour Implementation Plan

A focused plan for a team of 3 to turn the current TRVL app into a smarter, data-driven travel planner with OAuth email import, smarter upload parsing, and a trip-specific agent that proposes schedule changes.

| | |
|---|---|
| Team size | 3 people |
| Time budget | 3 hours each |
| Total effort | 9 engineer-hours |
| Target | Deployable MVP demo |
| Main rule | Review before save |

**Implementation principle:** OAuth emails, uploaded files, pasted text, and the trip agent should never write directly to the itinerary. They all create `ProposedTripChange` records first. Users review, edit, approve, and then TRVL saves the final itinerary items.

## 1. Current App Baseline

The current app already has the right foundations: Next.js App Router, Supabase auth and RLS, trip members, itinerary items, expenses, shared links, and an AI email import route. The main issue is integration shape: `app/api/ai/import-email/route.ts` currently parses pasted text and inserts itinerary items immediately. For the new feature set, that direct write should become a proposal workflow.

| Existing piece | Use it for | Change needed |
|---|---|---|
| `SmartImportPanel.tsx` | Entry point for paste/import UI | Turn into tabbed import surface: Email, Upload, Paste |
| `import-email/route.ts` | AI extraction baseline | Return proposals instead of directly inserting items |
| `itinerary_items` | Final approved calendar/trip data | Add source fields later; do not use as staging table |
| `lib/types.ts` | Shared contract | Add proposal, import job, connected account, and preferences types |

## 2. MVP Scope For 3 Hours Each

### Must Ship

- Shared proposal type
- Import review UI
- Upload route for PDF/image/text/HTML
- Gmail OAuth skeleton or working Gmail if env is ready
- Trip agent that proposes changes
- Approve proposals into existing itinerary items

### Nice If Time

- Outlook OAuth provider
- Duplicate detection
- Import history list
- Confidence badges
- Conflict warnings for overlaps

### Explicitly Defer

- Background inbox scanning
- Gmail push notifications
- Automatic schedule mutation
- Multi-provider production verification
- Long-term raw email storage

**OAuth reality check:** Gmail message-body access uses restricted Gmail scopes. For a 3-hour sprint, build user-selected import and a clean provider interface. Do not promise always-on mailbox monitoring. Google verification and security review can matter before public launch. Microsoft delegated `Mail.Read` is simpler for prototypes, but still needs clear consent and a disconnect flow.

## 3. Shared Contract

This contract is the integration center. Every feature returns this same shape, so the review UI and approval API do not care whether the source was Gmail, Outlook, PDF, screenshot, pasted text, or the trip agent.

```
export type ProposalSource = "gmail" | "outlook" | "upload" | "paste" | "agent";

export type ProposedTripChange = {
  id?: string;
  trip_id: string;
  source: ProposalSource;
  source_ref?: string | null;
  type: "flight" | "hotel" | "transport" | "activity" | "restaurant";
  title: string;
  detail?: string | null;
  day_date?: string | null;
  time?: string | null;
  end_time?: string | null;
  location_name?: string | null;
  latitude?: number | null;
  longitude?: number | null;
  booking_ref?: string | null;
  booking_url?: string | null;
  cost?: number;
  currency?: string;
  notes?: string | null;
  confidence: number;
  warnings: string[];
  action: "create" | "update" | "move" | "delete";
  target_item_id?: string | null;
};
```

## 4. Database Additions

Keep the current `itinerary_items` table as final approved data. Add staging and memory tables for imports and the agent.

| Table | Purpose | Key fields |
|---|---|---|
| `connected_accounts` | Stores user OAuth providers and encrypted tokens. | `user_id`, `provider`, `email`, `access_token_enc`, `refresh_token_enc`, `expires_at` |
| `import_jobs` | Tracks each paste/upload/email import attempt. | `trip_id`, `user_id`, `source`, `status`, `raw_summary`, `error` |
| `proposed_trip_changes` | Shared review queue for imports and agent suggestions. | `trip_id`, `import_job_id`, `payload jsonb`, `status`, `confidence` |
| `user_preferences` | Stores trip-agent personalization. | `user_id`, `pace`, `budget_style`, `food_notes`, `start_time`, `end_time` |
| `agent_messages` | Optional lightweight trip agent memory. | `trip_id`, `user_id`, `role`, `content`, `created_at` |

## 5. API Design

| Route | Owner | Purpose |
|---|---|---|
| `POST /api/imports/paste` | Backend + AI | Refactor current email import to create proposals. |
| `POST /api/imports/upload` | AI/import owner | Accept multipart file, parse PDF/image/HTML/text, create proposals. |
| `GET /api/oauth/google/start` | Backend/OAuth owner | Start Google consent flow. |
| `GET /api/oauth/google/callback` | Backend/OAuth owner | Exchange code, store account tokens, return to trip import modal. |
| `GET /api/email/search` | Backend/OAuth owner | Return likely travel confirmations for user selection. |
| `POST /api/email/import` | Backend + AI | Fetch selected email body and create proposals. |

| | | |
|---|---|---|
| `POST /api/proposals/[id]/approve` | Backend owner | Create/update/move/delete final itinerary item after review. |
| `POST /api/trips/[id]/agent` | Frontend/agent owner | Given chat input, itinerary, and preferences, create proposed changes. |

## 6. Team Split

### Person A: Backend + OAuth

**Owns** data model, API contracts, OAuth, approval endpoint.

Avoids touching upload parsing internals and most UI files.

### Person B: Smart Upload + Extraction

**Owns** file upload route, PDF/image/HTML/text normalization, AI extraction prompt.

Works against the shared proposal type and mock review endpoint if needed.

### Person C: Frontend + Trip Agent

**Owns** import modal, review queue UI, trip agent panel, apply/reject UX.

Uses mock proposals until backend routes are ready, then swaps endpoints.

## 7. Three-Hour Timeline

| Time | All-team integration action | Person A | Person B | Person C |
|---|---|---|---|---|
| 0:00-0:15 | Create branch, agree on contract, assign files. | Add shared types to `lib/types.ts`. | Copy proposal type into parser tests/examples. | Sketch UI states with mocked proposal data. |
| 0:15-0:35 | Merge contract first. Everyone pulls. | Add SQL for new tables and basic RLS. | Create `/api/imports/upload` route shell. | Refactor `SmartImportPanel` into tabs. |
| 0:35-1:15 | Keep API response shape stable. | Add proposal create/list/approve endpoints. | Implement text, HTML, and PDF extraction path. | Build review panel with edit, approve, reject. |
| 1:15-1:45 | First smoke integration: mock proposals to approved itinerary. | Add Gmail OAuth start/callback skeleton. | Add image path using AI vision or OCR fallback. | Add agent chat panel with mocked proposed moves. |
| 1:45-2:15 | Second smoke integration: upload creates proposals. | Add email search/fetch interface. Gmail first. | Connect upload route to extraction and proposal insert. | Connect review UI to real proposal endpoints. |
| 2:15-2:40 | Freeze features. Only integration and bugs. | Env checks, disconnect route, route guards. | Improve prompt, warnings, malformed file errors. | Connect agent route and proposal preview. |
| 2:40-3:00 | Run build, demo script, deployment checklist. | Supabase SQL and Vercel env checklist. | Test sample PDF/image/email text. | Demo flow polish and screenshots if needed. |

## 8. File Ownership To Avoid Merge Pain

| Person | Primary files | Shared files requiring coordination |
|---|---|---|
| Person A | `supabase/schema.sql`, `app/api/oauth/*`, `app/api/email/*`, `app/api/proposals/*` | `lib/types.ts`, `.env.example` |
| Person B | `app/api/imports/upload/route.ts`, `app/api/imports/paste/route.ts`, `lib/ai.ts`, new `lib/imports/*` | `package.json` if adding parser dependencies |
| Person C | `components/trip/SmartImportPanel.tsx`, new `ImportReviewPanel.tsx`, new `TripAgentPanel.tsx` | Trip page files where panels are mounted |

## 9. Implementation Details By Feature

### OAuth Email Import

- Build Gmail first. Add Outlook only if Gmail path is stable before 2:15.
- Use user-selected import: list likely confirmations, let the user choose, then parse selected messages.
- Search query examples: `confirmation OR itinerary OR reservation OR booking`, sender filters for airlines/hotels.
- Store tokens encrypted. If encryption is not ready in 3 hours, do not deploy OAuth to production; gate behind missing env.
- Add disconnect route that deletes `connected_accounts` rows.

### Smart Upload

- Support `.txt` and `.html` first because they are fast and reliable.
- PDF: extract embedded text; if empty, warn that screenshot/OCR path is needed.
- Images: use an AI vision extraction route or OCR dependency if already available.
- Normalize input into sections: subject, sender, visible text, tables, links, attachment names.
- Return proposal confidence and warnings, not only parsed fields.

### Trip-Specific Agent

- Agent reads current itinerary, trip dates, budget, and saved preferences.
- Agent can propose create/update/move/delete actions, but cannot directly mutate the trip.
- Initial commands to support: "make this less packed", "move this later", "add more food", "keep mornings free".
- Agent output is stored as proposals and appears in the same review panel as imports.

## 10. Integration Checkpoints

1. **0:15 contract checkpoint:** `ProposedTripChange` exists in `lib/types.ts`. Everyone imports it.
2. **1:15 save checkpoint:** A fake proposal can be approved into `itinerary_items`.
3. **2:15 import checkpoint:** Upload or paste creates proposals visible in review UI.
4. **2:40 freeze checkpoint:** Stop adding features. Fix integration and deploy blockers only.
5. **3:00 demo checkpoint:** One clean flow works from input to approved calendar item.

## 11. Deployment Checklist

### Before Build

- Run Supabase SQL additions.
- Add env vars to `.env.local` and Vercel.
- Confirm `OPENAI_API_KEY` is set.
- Confirm OAuth redirect URL matches deployed domain.
- Confirm no raw emails or local env files are committed.

### Commands

```
npm install
npm run build
npm run dev

# after deployment
# test login, trip page, upload import,
# review approve, and agent proposal
```

## 12. Demo Script

1. Create or open a Tokyo trip.
2. Open Smart Import and upload a sample hotel confirmation PDF or image.
3. Show extracted proposals with confidence and warnings.
4. Edit one field, approve it, and show the item in the itinerary/calendar.
5. Open the trip agent and say: `Make this day less packed and keep mornings relaxed.`
6. Review the agent's proposed schedule moves and approve one.
7. If OAuth is ready, connect Gmail and import one selected confirmation email. If not, show the gated OAuth UI and explain the provider interface is ready.

## 13. Risk Controls

| Risk | Control |
|---|---|

| | |
|---|---|
| OAuth takes longer than expected | Ship upload + paste + OAuth UI skeleton. Do not block the whole demo on provider auth. |
| AI parses incorrectly | Review screen is mandatory. Confidence and warnings make uncertainty visible. |
| Merge conflicts | Follow file ownership and merge the shared contract first. |
| Security concerns | Do not store raw email bodies long-term. Delete or summarize after proposals are created. |
| Deployment breaks from missing env | Feature-gate OAuth and agent calls when required env vars are missing. |

## 14. Environment Variables

```
OPENAI_API_KEY=...
NEXT_PUBLIC_SUPABASE_URL=...
NEXT_PUBLIC_SUPABASE_ANON_KEY=...
SUPABASE_SERVICE_ROLE_KEY=...

GOOGLE_CLIENT_ID=...
GOOGLE_CLIENT_SECRET=...
GOOGLE_REDIRECT_URI=https://your-domain.com/api/oauth/google/callback

MICROSOFT_CLIENT_ID=...
MICROSOFT_CLIENT_SECRET=...
MICROSOFT_REDIRECT_URI=https://your-domain.com/api/oauth/microsoft/callback

TOKEN_ENCRYPTION_KEY=...
```

## 15. Final Definition Of Done

> The sprint is successful if one complete path works end-to-end: **source data enters TRVL, proposals are generated, the user reviews them, approved changes become itinerary items, and the deployed app does not expose secrets or mutate trips without confirmation.**

Reference notes: Gmail scopes and data policy are documented by Google at https://developers.google.com/workspace/gmail/api/auth/scopes and https://developers.google.com/gmail/api/policy. Microsoft Graph mail permissions are documented at https://learn.microsoft.com/en-us/graph/permissions-reference.